

4. 신경망 학습

106~146 p

학습 : 훈련 data로부터 가중치 매개변수 최적값을 자동으로 획득

손실 함수 : 신경망이 학습할 수 있도록 해주는 지표

⇒ 손실 함수 결과값을 가장 작게 만드는 가중치 매개변수를 찾아야함

⇒ How can we find? 함수의 기울기를 활용하는 경사법 이용

4.1. 데이터에서 학습

= 신경망의 특징 :

가중치 매개변수의 값을 데이터를 보고 자동으로 결정

4.1.1. 데이터 주도적 학습

: 이미지의 특징 추출 → 특징 패턴을 ML로 학습

- 특징 : 입력 data를 정확하게 추출할 수 있도록 설계된 변환기를 포인팅
 - data → 벡터 → 분류기법 등으로 학습
 - 모아진 data로부터 규칙을 찾아내는 건 "기계"의 역할

Q. 이미지에서 5라는 숫자를 인식하는 프로그램,

손글씨를 보고 5인지 알아 맞추는 프로그램 구현을 원한다면 어떤 알고리즘을 써야 할까?

A. 특징을 추출하여 특징의 패턴을 기계학습 기술로 학습

규칙을 찾아내는 역할은 "기계" 담당

but 이미지 → 벡터 변환시 사용하는 특징은 사람이 설계

=> "어떤 정보를 추출하면 유용할까?"를 고민

그 정보를 숫자 벡터 형태로 뽑아주는 역할

⇒ 문제에 적합한 특징을 써야 좋은 결과를 낼 수 있다.

예시 : 물체 인식 (고양이 vs 개)

- 문제: 사진이 고양이인지 개인지 분류
- 특징 설계 예시:
 - 색상 히스토그램 (예: 갈색이 많은가?)
 -

4.1.2. 훈련 데이터와 시험 데이터

기계학습 문제의 데이터는

- a. 훈련 데이터
- b. 시험 데이터

1. 훈련 data만 사용하여 학습하면서 최적의 매개변수 찾기
2. 시험 데이터를 사용하여 학습(훈련) 모델 실력 평가

⇒ why 두개로 나누는가?

⇒ 범용 능력 (훈련에 쓰이지 X 데이터에서도 올바르게 풀어내는 능력) 을 평가하기 위함

⇒ 데이터 set 하나로만 학습했을 때 지나치게 최적화된 상태를 "오버피팅"이라고 함

4.2. 손실함수

신경망 학습에서는 **현재 상태를 하나의 "지표"**로 표현 한다.

또한 그 **"지표"**를 **가장 좋게** 만들어주는 가중치 매개변수의 값을 탐색한다.

⇒ **최적의 매개변수 값을 탐색**하는 것

⇒ 신경망에서 사용하는 지표는?

⇒ **"손실함수"**

⇒ 즉, 신경망 학습의 목표는?

손실함수를 최적의(가장 좋은) 가중치 매개변수의 값을 탐색

4.2.1. 오차제곱합

$$E = \frac{1}{2} \sum_k (y_k - t_k)^2$$

y_k 는 신경망의 출력(신경망이 추정한 값)

t_k 는 정답 레이블

k 는 데이터의 차원 수

```
y = [0.1, 0.05, 0.6, 0.0, 0.05, 0.1, 0.0, 0.1, 0.0, 0.0]
t = [0, 0, 1, 0, 0, 0, 0, 0, 0, 0]
```

추정한 정답 값:

입력 이미지가 2가 될 확률이 0.6으로 가장 높고 (soft-max 함수의 출력)

실제 정답 값:

3번째에 있는 값인 2가 정답 레이블로 지정

참고로 이렇게 한 원소만 1이고 나머지는 0으로 표기하는 것을 원-핫 인코딩이라고 부름

⇒ 위의 식 코드화

```
def sum_squares_error(y, t):
    return 0.5 * np.sum((y-t)**2)
```

```
[1] def sum_squares_error(y,t):  
    return 0.5*np.sum((y-t)**2)
```

```
[2] #정답은 2  
t = [0,0,1,0,0,0,0,0,0,0]
```

```
[3] #2일 확률이 높다고 추정(계산)한 결과  
y = [0.1, 0.05,0.6,0.0,0.05, 0.1, 0.0, 0.1,0.0, 0.0]
```

```
[4] import numpy as np  
sum_squares_error(np.array(y),np.array(t))
```

⇒ np.float64(0.097500000000000003)

```
[5] #7일 확률이 높다고 추정(계산)한 결과  
y = [0.1, 0.05, 0.1, 0.0, 0.05, 0.1, 0.0, 0.6, 0.0, 0.0]
```

```
[6] sum_squares_error(np.array(y),np.array(t))
```

⇒ np.float64(0.5975)

****손실함수가 작아야 정답에 가까운 것**

⇒ 오차제곱합 기준이기 때문, 오차가 더 작으니 정답과 가까운 것이다

⇒ 따라서 정답은 2로 추정했을때가 정답과 더 가까운 것

4.2.2. 교차 엔트로피 오차

$$E = -\sum_k t_k \log y_k$$

log는 밑이 e, 자연로그

정답 레이블은 2가 정답이라고 할 때, 신경망 출력이 0.6이라면?

⇒ 교차 엔트로피 오차 : $-\log 0.6 = 0.51$

정답 레이블은 2가 정답이라고 할 때, 신경망 출력이 0.1이라면?

⇒ 교차 엔트로피 오차 : $-\log 0.1 = 2.30$

⇒ 정답일 때 출력이 작아지면 오차는 커진다.

```
>>> t = [0, 0, 1, 0, 0, 0, 0, 0, 0, 0]
>>> y = [0.1, 0.05, 0.6, 0.0, 0.05, 0.1, 0.0, 0.1, 0.0, 0.0]
>>> cross_entropy_error(np.array(y), np.array(t))
0.51082545709933802
>>>
>>> y = [0.1, 0.05, 0.1, 0.0, 0.05, 0.1, 0.0, 0.6, 0.0, 0.0]
>>> cross_entropy_error(np.array(y), np.array(t))
2.3025840929945458
```

여기서 **결과(오차값)이 더 작은** 첫번째 추정이 **정답일** 가능성이 높은 것

4.2.3. 미니배치 학습

$$E = -\frac{1}{N} \sum_n \sum_k t_{nk} \log y_{nk}$$

data가 N개 일 때 t_{nk} 는 n번째 데이터의 k번째 값

손실함수를 구한 다음 그 값을 최대한 줄여주는 매개 변수를 찾아야하는데

모든 훈련 데이터를 대상으로 손실 함수를 구해야함

⇒ 훈련 데이터 100개가 있다면 그로부터 계산된 100개의 손실함수 값의 합을 지표로

⇒ 언제든 통일된 지표를 얻을 수 있다 "평균"을 얻기 때문

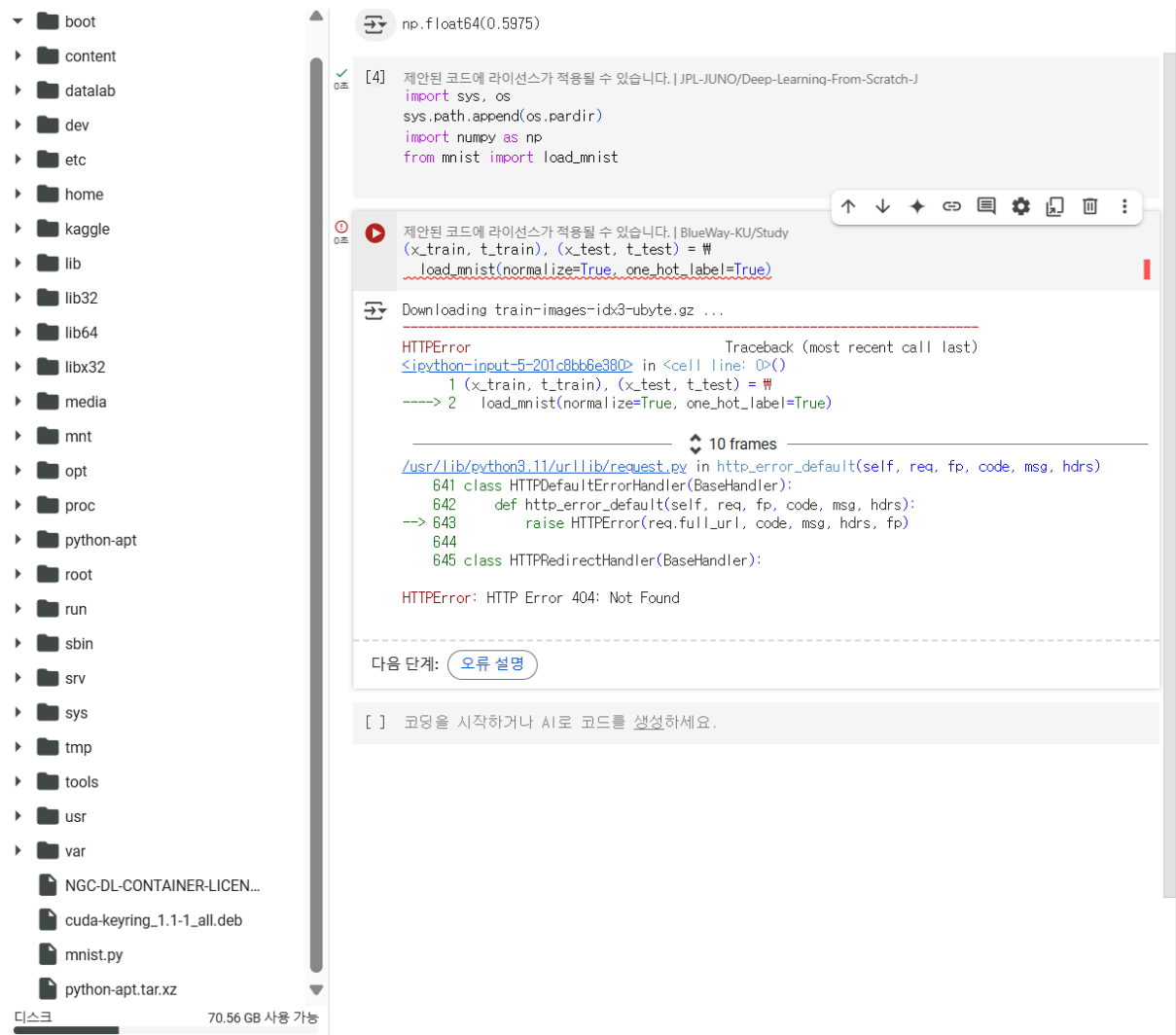
⇒ but MNIST 데이터셋은 훈련 데이터가 6만개

⇒ 모든 데이터가 아닌 일부를 추려 '근사치'로 이용

⇒ 일부만 골라 학습을 수행하는데

⇒ 이때의 "일부" (= "미니배치")

⇒ 일부만 무작위로 뽑아서 해당 데이터만 사용하여 학습



직접 실습_에러

```
train_size = x_train.shape[0]
batch_size = 10
batch_mask = np.random.choice(train_size, batch_size)
x_batch = x_train[batch_mask]
t_batch = t_train[batch_mask]
```

훈련 데이터에서 무작위로 10장을 뽑는 방법:

numpy random choice를 사용하면 된다

**** np.random.choice(데이터 갯수, 선정 갯수)**

```
>>> np.random.choice(60000, 10)
array([ 8013, 14666, 58210, 23832, 52091, 10153, 8107, 19410, 27260, 21411])
```

** 이렇게 되면 손실 함수도 미니배치로 계산한다

4.2.4. 교차 엔트로피 오차 구현하기

⇒ 미니배치 등의 배치 데이터를 지원하는 교차 엔트로피 오차 구현하기

```
def cross_entropy_error(y, t):  
    if y.ndim == 1:  
        t = t.reshape(1, t.size)  
        y = y.reshape(1, y.size)  
  
    batch_size = y.shape[0]  
    return -np.sum(t * np.log(y + 1e-7)) / batch_size
```

⇒ y의 차원에 따라 형상 변경 가능

⇒ 이전까지는 입력 data가 어떤 값일 확률이 높을지를 계산하는 것이었다.

⇒ 여기서는 정답 레이블은 원-핫인 맞다 아니다가 아니라 숫자 레이블로 주어졌을 때를 보임

ex) '2', '7', ...

```
def cross_entropy_error(y, t):  
    if y.ndim == 1:  
        t = t.reshape(1, t.size)  
        y = y.reshape(1, y.size)  
  
    batch_size = y.shape[0]  
    return -np.sum(np.log(y[np.arange(batch_size), t] + 1e-7)) / batch_size
```

4.2.5. 손실함수를 정의하는 이유

정확도가 아닌 반대의 손실 함수 값을 택하는 이유는?

⇒ **"미분"**

⇒ 최적의 매개변수를 탐색할때, 손실함수의 값을 '**가능한 작게**' 하는 매개변수 값을 find

⇒ 이때 매개변수의 미분(=기울기)을 계산하고 그를 단서로 매개변수의 값을 갱신

⇒ 손실함수의 미분이 "음수"

⇒ 가중치 매개변수를 양의 방향으로 변화

⇒ 손실함수의 값을 줄일 수 있다.

⇒ 손실함수의 미분이 "양수"

⇒ 가중치 매개변수를 음의 방향으로 변화

⇒ 손실함수의 값을 줄일 수 있다.

⇒ 손실함수의 미분이 "0"

⇒ 갱신의 stop part

정확도로 삼으면 매개변수의 미분이 대부분의 장소에서 0이 되기 때문

⇒ 정확도는 불연속적인 띄엄띄엄한 값으로 나타남 (소수점 X)

⇒ but 손실함수는 연속적인 수치로 변화함

⇒ So, 정확도는 작은 변화에 거의 반응 X, 반응이 불연속적으로 변화

⇒ 계단 함수를 활성화 함수로 사용하지 않는 이유라도 맞음

p.123

4.3. 수치 미분

⇒ 미분이란? 한 순간의 변화량

$$\frac{df(x)}{dx} = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}$$

→


```
def numerical_diff(f, x):
    h = 1e-50
    return (f(x + h) - f(x)) / h
```

4.3.1. 미분

'특정 순간'의 **변화량**

⇒ so, 미분은 한순간의 변화량을 표시한 것

Q. 다음 코드에서 개선되어야 할 것은?

```
# 나쁜 구현 예
def numerical_diff(f, x):
    h = 1e-50
    return (f(x + h) - f(x)) / h
```

A. 1) 반올림 오차 문제를 일으키기 때문에 최종 계산에 오차 발생

⇒ $1e-50$ 은 소수점 아래 50자리라는 뜻이기 때문

```
>>> np.float32(1e-50)
0.0
```

밀부분 삭제

⇒ 개선 방법 : 10^{-4} 이용하기

```
>>> np.float32(1e-50)
0.0
```

2) f의 차분

f 함수에서 미지수 $x+h$ 와 x 간의 차분을 구하고 있음

기본적인 미분은 x 위치에서 함수의 기울기 (=접선)

but, 여기서 $x+h$ 와 x 사이의 기울기임 ⇒ 한 점에서 기울기가 아니라는 뜻

so, 엄밀히 말하면 일치하진 않아 h 의 값을 무한히 0으로 좁히는 게 불가능

** 참고로 $x+h$ 와 $x-h$ 일때 차분을 계산하기도 하는데 이는 (중심/중앙)차분

4.3.2. 수치 미분의 예시

*간단한 함수 미분

$$y = 0.01x^2 + 0.1x$$

```
[1] def function_1(x):  
    return 0.01*x**2 + 0.1 *x
```

```
[2] import numpy as np  
    import matplotlib.pyplot as plt
```

```
▶ x = np.arange(0.0, 20.0, 0.1)  
  y = function_1(x)  
  plt.xlabel("x")  
  plt.ylabel("f(x)")  
  plt.plot(x,y)  
  plt.show()
```

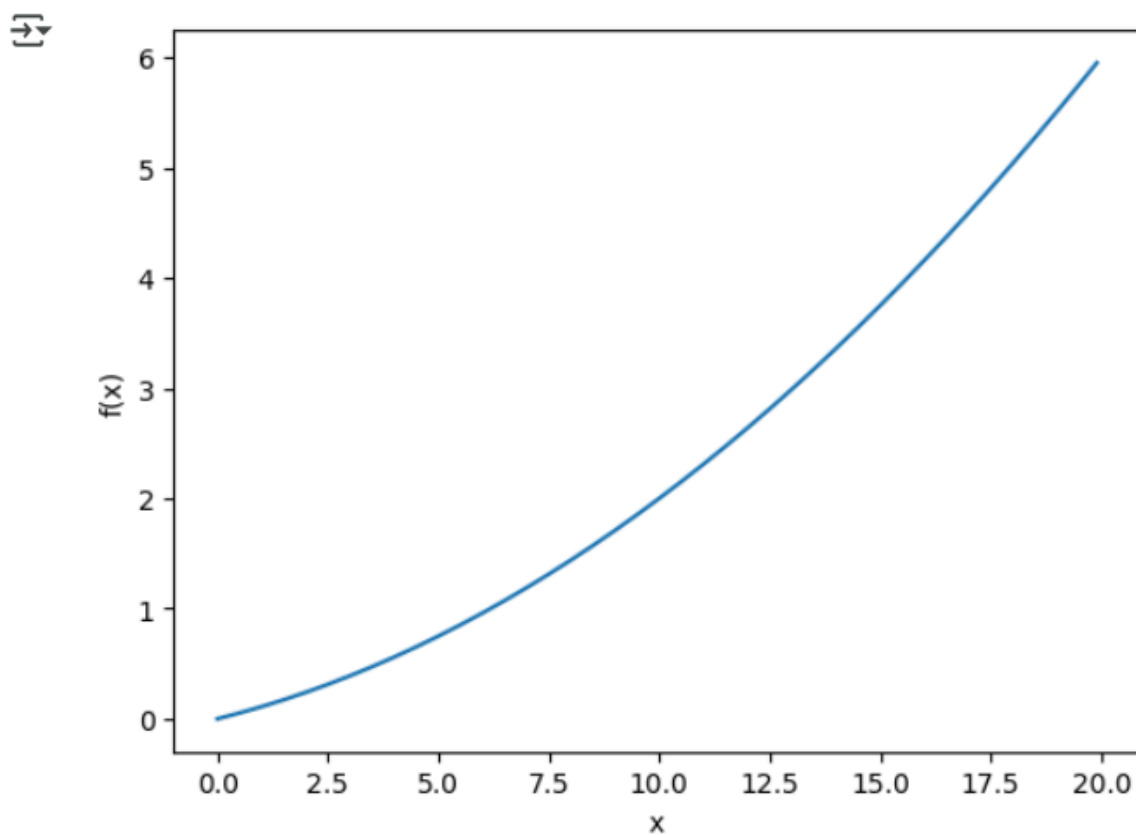
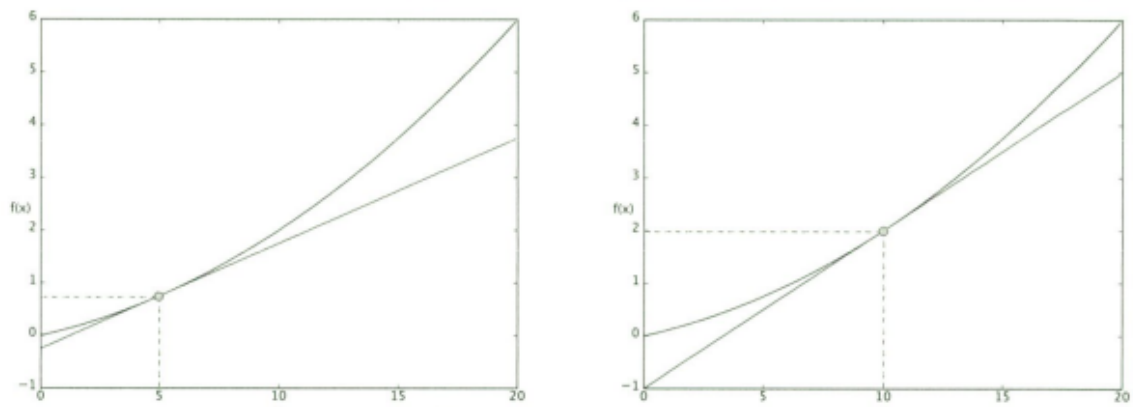


그림 4-7 $x = 5, x = 10$ 에서의 접선 : 직선의 기울기는 수치 미분에서 구한 값을 사용하였다.



4.3.3. 편미분

$$f(x_0, x_1) = x_0^2 + x_1^2$$

** 단순한 식이지만 변수가 2개라는 차이

⇒ 변수가 여럿인 함수에 대한 미분은 “편미분”

4.4. 기울기

변수와 편미분 계산을 동시에 수행하는 것

```

def numerical_gradient(f, x):
    h = 1e-4
    grad = np.zeros_like(x)

    for idx in range(x.size):
        tmp_val = x[idx]
        x[idx] = tmp_val + h

        fxh1 = f(x)

        x[idx] = tmp_val - h
        fxh2 = f(x)

        grad[idx] = (fxh1 - fxh2) / (2*h)
        x[idx] = tmp_val
    return grad

```

```

[20] numerical_gradient(function_2, np.array([3.0,4.0]))
File "<ipython-input-20-69c2ecec8a>", line 1
    numerical_gradient(function_2, np.array([3.0,4.0]))
                                             ^
SyntaxError: incomplete input

```

⇒ 오류

4.4.1. 경사법(경사 하강법)

→ 최적의 매개변수를 학습 시에 찾는 것

⇒ 기울기를 이용하여 찾을 것

⇒ but 지역 최소값에 있을 수 있고, 그래도 0 이기 때문에..

⇒ "고원"이라는 정체기에 빠진다,

⇒ So, 경사법은 일정거리만큼 기울어진 방향으로 이동하며 반복하는 것

⇒ 최소값을 찾는 건 경사 하강법, 최대값을 찾는 건 경사 상승법

$$x_0 = x_0 - \eta \frac{\partial f}{\partial x_0}$$

$$x_1 = x_1 - \eta \frac{\partial f}{\partial x_1}$$

밑줄친 부분은 학습률

경사 하강법 코드화

```
def gradient_descent(f, init_x, lr=0.01, step_num=100):
    x = init_x

    for i in range(step_num):
        grad = numerical_gradient(f, x)
        x -= lr * grad
    return x
```

- init_x : 초기값
- lr : learning rate
- step_num : 경사법 반복횟수
- numerical_gradient(f,x) : 함수의 기울기

4.4.2. 신경망에서 기울기

** 기울기 : 가중치 매개변수에 대한 손실 함수의 기울기

가중치 W, 손실함수 L인 신경망

이때의 경사는 @L/@W ⇒ 원소에 관한 "편미분"



```

import sys
import os
import numpy as np
#sys.path.append(os.pardir)
from functions import softmax, cross_entropy_error
from gradient import numerical_gradient

# 4.4.2 신경망에서의 기울기
class simpleNet:
    """docstring for simpleNet"""
    def __init__(self):
        self.W = np.random.randn(2, 3) # 정규분포로 초기화

    # 예측

    def predict(self, x):
        return np.dot(x, self.W)

    # 손실함수

    def loss(self, x, t):
        z = self.predict(x)
        y = softmax(z)
        loss = cross_entropy_error(y, t)

        return loss

net = simpleNet()
print(net.W) # 가중치 매개변수(랜덤)

```



```

[[-0.18006644  1.79242699 -0.64882541]
 [-2.24908767  0.15784346 -0.64398795]]

```

```

[28] x = np.array([0.6, 0.9])
     p = net.predict(x)

```

```

print(p)

```



```

[-2.13221877  1.2175153  -0.9688844 ]

```

⇒ 객체의 가중치 확인

```
✓ [28] x = np.array([0.6, 0.9])  
0초 p = net.predict(x)
```

```
print(p)
```

```
⇒ [-2.13221877  1.2175153 -0.9688844 ]
```

```
✓ [29] print(np.argmax(p)) # 최댓값의 인덱스  
0초
```

```
⇒ 1
```

```
✓ [30] t = np.array([0, 0, 1]) # 정답 레이블  
0초 print(net.loss(x, t))
```

```
⇒ 2.323910504571879
```

⇒ 입력 x 를 넣어서 예측값 p 를 계산

⇒ $\text{np.argmax}(p)$ 는 예측 클래스

⇒ 예측값과 정답 레이블을 비교하여 loss 계산

```
✓ [31]  
0초  
  
def f(W):  
    return net.loss(x, t)  
  
dW = numerical_gradient(f, net.W)  
print(dW)  
  
⇒ [[ 0.01835101  0.52291496 -0.54126598]  
    [ 0.02752652  0.78437245 -0.81189897]]
```

⇒ 기울기 계산

⇒ W 의 변화에 따른 기울기 계산

⇒ 경사법에 따른 가중치 매개변수 갱신

4.5. 학습 알고리즘 구현하기

전제 :

신경망에는 적응 가능한 가중치, 편향이 있다

이 가중치와 편향을 훈련 데이터에 맞게 조정해주는 것이 '학습'이다

⇒ 신경망 학습 절차는 다음과 같다

< 확률적 경사 하강법 >

1. 미니배치

- a. 훈련 데이터 중 일부를 무작위로 가져온다
- b. 선별된 데이터를 "미니배치"라고 부른다
- c. 선별된 data인 미니 배치 손실 함수 값을 줄여야한다
- d. 손실함수란?

⇒ **모델의 예측값과 정답값(실제값)** 사이의 ****차이(오차)****를 수치적으로 표현한 함수

2. 기울기 산출

- a. 미니배치의 **손실 함수 값을 줄이기** 위해 각 가중치 매개변수 기울기 구하기
- b. **기울기는 손실 함수의 값을 가장 작게하는 방향 제시**
- c. 기울기가 0에 가까워질수록 학습이 수렴

3. 매개변수 갱신

- a. 가중치 매개변수는 기울기 방향으로 조금씩 갱신한다

4. 반복

- a. 1~3단계 반복하기

⇒ 딥러닝 프레임 워크에서 **SGD**라는 함수로 이 모든 기능을 구현하고 있다.

4.5.1. 2층 신경망 클래스 구현

⇒ 2층 신경망 하나의 클래스로 구현하기

```

[38] # coding: utf-8
import sys, os
import numpy as np
sys.path.append(os.pardir) # 부모 디렉터리의 파일을 가져올 수 있도록 설정
from functions import *
from gradient import numerical_gradient

class TwoLayerNet:

    def __init__(self, input_size, hidden_size, output_size, weight_init_std=0.01):
        # 가중치 초기화
        self.params = {}
        self.params['W1'] = weight_init_std * np.random.randn(input_size, hidden_size)
        self.params['b1'] = np.zeros(hidden_size)
        self.params['W2'] = weight_init_std * np.random.randn(hidden_size, output_size)
        self.params['b2'] = np.zeros(output_size)

    def predict(self, x):
        W1, W2 = self.params['W1'], self.params['W2']
        b1, b2 = self.params['b1'], self.params['b2']

        a1 = np.dot(x, W1) + b1
        z1 = sigmoid(a1)
        a2 = np.dot(z1, W2) + b2
        y = softmax(a2)

        return y

    # x : 입력 데이터, t : 정답 레이블
    def loss(self, x, t):
        y = self.predict(x)

        return cross_entropy_error(y, t)

    def accuracy(self, x, t):
        y = self.predict(x)
        y = np.argmax(y, axis=1)
        t = np.argmax(t, axis=1)

        accuracy = np.sum(y == t) / float(x.shape[0])
        return accuracy

    # x : 입력 데이터, t : 정답 레이블
    def numerical_gradient(self, x, t):
        loss_W = lambda W: self.loss(x, t)

        grads = {}
        grads['W1'] = numerical_gradient(loss_W, self.params['W1'])
        grads['b1'] = numerical_gradient(loss_W, self.params['b1'])
        grads['W2'] = numerical_gradient(loss_W, self.params['W2'])
        grads['b2'] = numerical_gradient(loss_W, self.params['b2'])

        return grads

    def gradient(self, x, t):
        W1, W2 = self.params['W1'], self.params['W2']
        b1, b2 = self.params['b1'], self.params['b2']
        grads = {}

        batch_num = x.shape[0]

        # forward
        a1 = np.dot(x, W1) + b1
        z1 = sigmoid(a1)
        a2 = np.dot(z1, W2) + b2
        y = softmax(a2)

        # backward
        dy = (y - t) / batch_num # ???
        grads['W2'] = np.dot(z1.T, dy)
        grads['b2'] = np.sum(dy, axis=0)

        da1 = np.dot(dy, W2.T)
        dz1 = sigmoid_grad(a1) * da1
        grads['W1'] = np.dot(x.T, dz1)
        grads['b1'] = np.sum(dz1, axis=0)

        return grads

```

TwoLayerNet의 메서드:

init는 클래스 초기화

⇒ 입력층의 뉴런 수, 은닉층의 뉴런 수, 출력층의 뉴런 수

MNIST 인식에서는 784(28X28), 출력 10

** 은닉층의 갯수인 hidden_size는 적당히

4.5.2. 미니배치 학습 구현하기

경사법에 의한 갱신 횟수를 1000번으로 설정하고 갱신할 때마다 손실 함수를 계산한다.

```
"""
(x_train, t_train), (x_test, t_test) = #
    load_mnist(normalize=True, one_hot_label=False)

# 하이퍼 파라미터
iters_num = 1000 # 반복횟수
train_size = x_train.shape[0]
batch_size = 100 # 미니배치 크기
learning_rate = 0.1
network = TwoLayerNet(input_size=784, hidden_size=50, output_size=10)

train_loss_list = []
train_acc_list = []
test_acc_list = []

# 1에폭당 반복 수
iter_per_epoch = max(train_size / batch_size, 1)

for i in range(iters_num):
    print(i)
    # 미니배치 획득
    batch_mask = np.random.choice(train_size, batch_size)
    x_batch = x_train[batch_mask]
    t_batch = t_train[batch_mask]

    # 기울기 계산
    grad = network.numerical_gradient(x_batch, t_batch)
    # grad = network.gradient(x_batch, t_batch) # 다음 장에서 구현할 더 빠른 방법!

    # 매개변수 갱신
    for key in ('W1', 'b1', 'W2', 'b2'):
        network.params[key] -= learning_rate * grad[key]

    # 학습 경과 기록
    loss = network.loss(x_batch, t_batch)
    train_loss_list.append(loss)
```



```
ModuleNotFoundError                                Traceback (most recent call last)
<ipython-input-42-97a7b1db77c1> in <cell line: 0>()
      4 sys.path.append(os.pardir)
      5 from mnist import load_mnist
----> 6 import TwoLayerNet
      7
      8
```

ModuleNotFoundError: No module named 'TwoLayerNet'

NOTE: If your import is failing due to a missing package, you can manually install dependencies using either !pip or !apt.

To view examples of installing some common dependencies, click the "Open Examples" button below.

⇒ Q. 있는데 없다고 뜬

4.5.3. 시험 데이터로 평가하기

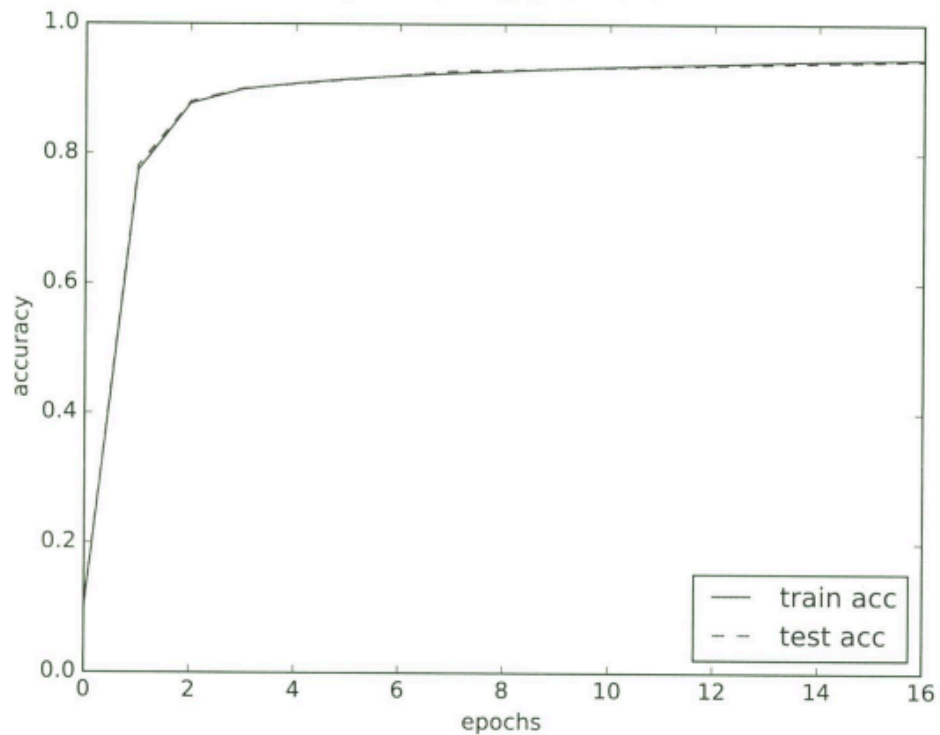
손실 함수의 값이 점점 내려가야해 ⇒ 잘 학습되고 있는 것

오버피팅 주의 **

⇒ 훈련 데이터 외에도 잘 인식하는지 확인

⇒ 신경망 학습의 목표는 범용적인 능력을 익히는 것이기 때문

그림 4-12 훈련 데이터와 시험 데이터에 대한 정확도 추이



⇒ 훈련 데이터와 시험 데이터가 학습 될수록 (에폭이 증가할수록) 정확도가 높아진다

⇒ and 두 정확도에는 차이가 없음

⇒ 오버피팅이 일어나지 않았다는 결론

4.6. 정리

- 신경망 학습의 목표:
 - 손실 함수를 기준으로 값이 가장 작아지는 가중치 매개변수 값을 찾아내는 것
 - 그러한 방법으로는 "경사법" 이 있으며, 이는 함수의 기울기를 이용하는 방법